

A developer who digs into root causes and enjoys solving difficult problems

Backend Engineer · AI Application Developer

kkuldangi2@gmail.com [GitHub ↗](#) [Blog ↗](#) [LinkedIn ↗](#)

SUMMARY

PDF conversion pass rate: 45.17% → 89.89%

RAG evidence coverage: 66.25% → 91.25%

Drug description conversion: \$200 estimated → \$11.18 spent

PROJECTS & EXPERIENCE

Fruition

2026.04 — Present

LLM Wiki-based AI workspace · AI SW Maestro, 17th cohort

Team project / Lead for all AI features

Teammates built the frontend and backend. I proposed the initial MSA design and finalized service boundaries, data ownership and communication with teammates and mentors.

[GitHub · local-pilot ↗](#) [GitHub · AI repository ↗](#)

Converting and analyzing uploaded documents

Extracted and repaired PDFs for retrieval and grounded answers, and reduced waiting time when processing multiple documents.

- PDF → Markdown conversion
Problem: Open-source conversion damaged tables and equations; full AI conversion still had errors and high API usage.
Solution: Compared PDF tools and adopted AnyDoc. Repaired damaged text with AI, processed tables and equations separately, and preserved original figures.
Result: Internal pass rate on 30 PDF pages improved from 45.17% to 89.89%.
- Parallel Wiki generation
Problem: Later documents waited for earlier external AI responses.
Solution: Compared worker counts and requests per worker. Capped workers at 4 to manage memory; rejected extra concurrency and GPU parallelism that slowed processing.
Result: Analysis of 4 documents fell from 282.11s to 73.89s (about 74% less).

Improving retrieval for grounded AI answers

Retrieved relevant passages from stored documents for AI answers. Improved evidence coverage and recognition of questions the documents could not answer.

- Evidence selection for RAG
Problem: Retrieval missed required facts and returned evidence for unanswerable questions.
Solution: Applied Jev after routing tests, then controlled candidates and return limits after a full-corpus trial. Tuned to 300 candidates and up to 4 parallel batches.
Result: Across 9 papers, coverage on 80 answerable questions rose from 66.25% to 91.25%. False evidence returns on 20 unanswerable questions fell from 20 to 0. Median time before/after Jev tuning: 9.090s → 2.251s.

AI Agent and reusable Skills

Built natural-language document editing and folder organization, with changes applied after user approval and recurring tasks saved as reusable instructions.

- User-defined task instructions (Skills)
Problem: Developers cannot build every recurring workflow users need.
Solution: Built drafting, review and publishing of natural-language instructions, with checks against bypassing permissions or approval. Document changes require separate preview and approval.
Result: Published Skills became available to the Agent. In a 6-document check, no changes occurred before approval; 3 folders were created and documents moved afterward.
- Document editing quality
Problem: Some generated drafts did not fulfill editing intent.
Solution: Improved how edit goals and retry inputs were passed, then added semantic evaluation and at most one rewrite.
Result: Passes on the same 114 drafts increased from 94 to 104.

Service responsibilities and communication

Defined responsibilities, data ownership and communication for services including AI. Proposed the initial design and finalized it with teammates and mentors.

- Proposed the initial MSA design, then jointly finalized service boundaries, data ownership and communication with teammates and mentors.

Internal evaluations. RAG timing compares Jev before/after tuning. Parallel document timing covers analysis only, excluding storage and post-processing.

See the detailed portfolio for the problem-solving process and experimental results.

[PDF conversion ↗](#) [Parallel document processing ↗](#) [Agent Skill design ↗](#) [RAG evidence selection ↗](#)

PROJECTS & EXPERIENCE

Pilltip

2025.03 — 2025.12

Personalized AI medication service · Pusan National University
Team project / Backend and AI application development

Teammates owned Android, questionnaires, user DB modeling, login and other user features, and security including AES-GCM. The features below are my work.

[GitHub repository](#)

Drug data preparation pipeline

Rewrote about 44,000 drug records in plain language. Converted repeated explanations once and mapped them back to each drug to reduce cost.

- Problem: Converting all 44,000 drug records was estimated to cost about \$200 in API calls.
Solution: Removed exact duplicates, then semantically equivalent sentences. Filled the missing 5% with ingredient-specific precautions, converted via GPT Batch API and mapped results back to each drug.
Result: Conversion input shrank from 1GB to 326MB. Actual API spending was \$11.18 versus about \$200 estimated for all original text.

Estimated cost versus actual spending; preprocessing labor is excluded.

Medication chatbot with health-profile personalization

Found drug and medication information from natural-language symptoms. Evaluated health profiles internally, separately from information sent to external AI.

- Problem: Everyday symptom language differed from drug indications, while sensitive health data needed to stay out of external AI inputs.
Solution: Used Weaviate semantic search and AI orchestration for drug discovery, medication risks and dosage. Evaluated pregnancy, current medications and underlying conditions in internal code.
Result: Implemented personalized symptom-based search while excluding those profile fields from external LLM inputs.

Backend implementation

Built server features for drug search, risk information, medication reminders and logs, and family profiles. Integrated with teammates responsible for the mobile app, users and security.

- Designed the drug DB and implemented drug search, autocomplete and Drug Utilization Review (DUR) risk information with Java and Spring Boot.
- Implemented FCM medication reminders and logs, deep-link friend invitations, and family profile switching.
- Shared API documentation, meeting notes and architecture decision records in Notion.

See the detailed portfolio for the problem-solving process and experimental results.

[Data pipeline case study](#) [Personalization case study](#)

Activities & contributions

2026.06 — 2026.06

Rhwp open-source contributions

Investigated bugs, submitted fixes and addressed CI in a Rust HWP/HWPX project. Fixed textFlow attributes of images, tables and shapes being reset after saving.

[GitHub repository](#) [PR #1213 · Merged](#) [PR #1351 · Submitted](#)

2025.03 — 2026.01



APPTIVE, Pusan National University

Backend mentee and mentor. Led 6 mentoring and code-review sessions for 12 mentees. Helped revise the curriculum to strengthen HTTP, Servlet, REST API and database foundations.

APPTIVE Backend Mentor Merit Award · 2026.01.09

2025.11 / 2025.07



SK AI SUMMIT · K-ICT WEEK in Busan

Represented Pusan National University at the Pilltip booth, demonstrating the service and answering technical questions.

Skills

Backend	Java · Spring Boot · Spring AI · Python Authentication and authorization (Spring Security, OAuth 2.0) Personal shopping project · Login ↗ Todo web app · OAuth ↗
AI & Search	LangChain · LangGraph · RAG · LLM Evaluation · Jev Elasticsearch · Weaviate · BGE-M3 · BM25
Data & Infra	MySQL · PostgreSQL · Redis · Kafka Docker · Docker Compose · GitHub Actions

Education & certification

2022.02 — 2026.02 (Transfer)	Pusan National University GPA 4.12 / 4.5
2019.03 — 2022.02 (Withdrew)	Daegu University GPA 4.2 / 4.5
2025.09	Engineer Information Processing Human Resources Development Service of Korea

Awards

2025.10		Capstone Design Gold Award Software & AI division / College of Information and Biomedical Engineering, Pusan National University
2025.09		Busan DATA WEEK Top Excellence Award Data utilization case competition / Busan Techno Park
2025.09		AI LAUNCH Career School Startup Hackathon Encouragement Award KRYPTON X x Root Impact x Google.org
2025.08		SW University Digital Competition Sponsor Award SW University Council